

SATELLITE FLIGHT COMPUTER

COMPLETE CODE REFERENCE

Document 3 of 4 — Every source file, complete and in order

FILES IN THIS DOCUMENT

#	File Path	Purpose
1	src/common/flight_types.h	Shared data structures, enums, and constants used by every task.
2	src/FreeRTOSConfig.h	Master configuration file for the FreeRTOS kernel.
3	linker.ld	Memory map for the QEMU lm3s6965evb machine (ARM Cortex-M3).
4	src/startup.c	Runs before main().
5	src/drivers/uart_driver.h	Public API for the UART driver.
6	src/drivers/uart_driver.c	Hardware UART driver targeting UART0 on the LM3S6965 (QEMU).
7	src/drivers/watchdog.h	Watchdog timer API.
8	src/drivers/watchdog.c	Software watchdog using a FreeRTOS one-shot timer.
9	src/main.c	Entry point.
10	src/tasks/adcs_task.c	Attitude Determination & Control System.
11	src/tasks/telemetry_task.c	Downlink task running at 1 Hz.
12	src/tasks/health_monitor.c	Runs at 10 Hz with the highest priority.
13	src/tasks/command_handler.c	Receives single-byte commands from the ground station GUI via UART.
14	CMakeLists.txt	Root build file.
15	ground_station/telemetry_parser.py	Python module that parses raw binary UART bytes into telemetry packet dicts.
16	ground_station/gui.py	Tkinter + matplotlib ground station GUI.
17	tests/test_parser.py	pytest unit tests for the telemetry parser.

Each file below is shown with its **exact file path** at the top, a one-paragraph description of what it does and why, and then the **complete, ready-to-copy code**. No file is partial. No file requires you to fill in any blanks.

FILE 1 OF 17

src/common/flight_types.h

Shared data structures, enums, and constants used by every task. Define this first — it is the contract between all parts of the system.

```
#ifndef FLIGHT_TYPES_H
#define FLIGHT_TYPES_H

#include <stdint.h>
#include <stdbool.h>

/* =====
ATTITUDE STATE
Written by: adcs_task
Read by:   telemetry_task, health_monitor, command_handler
===== */
typedef struct {
    float    quaternion[4]; /* Attitude quaternion [w, x, y, z] */
    float    omega[3];      /* Angular velocity [rad/s] */
    float    euler[3];      /* Roll, Pitch, Yaw [degrees] */
    uint32_t timestamp_ms;   /* Mission elapsed time in milliseconds */
    uint8_t  mode;          /* Current operating mode (FlightMode_t) */
} ADCSState_t;

/* =====
TELEMETRY PACKET (binary wire format sent over UART)
__attribute__((packed)) = no padding bytes between fields.
This MUST match the struct.unpack format in telemetry_parser.py
===== */
#define TLM_MAGIC 0xA5C3

typedef struct __attribute__((packed)) {
    uint16_t magic;          /* Always TLM_MAGIC — marks packet start */
    uint16_t sequence;      /* Incrementing counter, detects drops */
    uint32_t timestamp_ms;
    float    roll_deg;
    float    pitch_deg;
    float    yaw_deg;
    float    omega_x;
    float    omega_y;
    float    omega_z;
    uint8_t  mode;
    uint8_t  fault_flags;   /* Bitmask — see FAULT_* defines below */
    int16_t  temperature;  /* Degrees C * 10 e.g. 235 = 23.5 C */
    uint16_t checksum;     /* XOR of all preceding bytes */
} TelemetryPacket_t;

/* =====
FAULT FLAGS (bitmask stored in fault_flags field)
Multiple faults can be active simultaneously.
===== */
#define FAULT_NONE          0x00
#define FAULT_ADCS_TIMEOUT 0x01 /* ADCS missed its heartbeat */
#define FAULT_SENSOR_INVALID 0x02 /* Sensor reading out of range */
#define FAULT_WATCHDOG      0x04 /* Watchdog nearing expiry */
#define FAULT_STACK_LOW     0x08 /* A task stack is nearly full */
#define FAULT_QUEUE_FULL    0x10 /* Telemetry queue overflowed */

/* =====
OPERATING MODES
===== */
```

```
===== */
typedef enum {
    MODE_SAFE      = 0,  /* Emergency: minimal ops, await ground command */
    MODE_DETUMBLE  = 1,  /* Reduce rotation after launch */
    MODE_NADIR     = 2,  /* Point camera/antenna toward Earth */
    MODE_NOMINAL   = 3,  /* Full science/operations mode */
} FlightMode_t;

#endif /* FLIGHT_TYPES_H */
```

FILE 2 OF 17

src/FreeRTOSConfig.h

Master configuration file for the FreeRTOS kernel. Every tunable parameter lives here. Must be in an include path visible to all FreeRTOS source files.

```
#ifndef FREERTOS_CONFIG_H
#define FREERTOS_CONFIG_H

/* ---- Core scheduler ---- */
#define configUSE_PREEMPTION 1
#define configUSE_TIME_SLICING 1
#define configUSE_IDLE_HOOK 1 /* vApplicationIdleHook() in main.c */
#define configUSE_TICK_HOOK 0
#define configCPU_CLOCK_HZ ((unsigned long) 12000000)
#define configTICK_RATE_HZ ((TickType_t) 1000) /* 1 ms tick */
#define configMAX_PRIORITIES 8
#define configMINIMAL_STACK_SIZE ((uint16_t) 128)
#define configTOTAL_HEAP_SIZE ((size_t) (20 * 1024)) /* 20 KB */
#define configMAX_TASK_NAME_LEN 16
#define configUSE_16_BIT_TICKS 0
#define configIDLE_SHOULD_YIELD 1

/* ---- Synchronisation primitives ---- */
#define configUSE_MUTEXES 1 /* REQUIRED for priority inheritance */
#define configUSE_RECURSIVE_MUTEXES 1
#define configUSE_COUNTING_SEMAPHORES 1
#define configUSE_QUEUE_SETS 0

/* ---- Software timers (used by watchdog) ---- */
#define configUSE_TIMERS 1
#define configTIMER_TASK_PRIORITY (configMAX_PRIORITIES - 1)
#define configTIMER_QUEUE_LENGTH 10
#define configTIMER_TASK_STACK_DEPTH 256

/* ---- Debug and safety ---- */
#define configUSE_TRACE_FACILITY 1
#define configUSE_STATS_FORMATTING_FUNCTIONS 1
#define configCHECK_FOR_STACK_OVERFLOW 2 /* 2 = full check, calls hook */
#define configUSE_MALLOC_FAILED_HOOK 1 /* calls vApplicationMallocFailedHook */
#define configASSERT(x) if((x) == 0) { taskDISABLE_INTERRUPTS(); for(;;); }

/* ---- API includes ---- */
#define INCLUDE_vTaskDelay 1
#define INCLUDE_vTaskDelayUntil 1 /* REQUIRED for fixed-rate tasks */
#define INCLUDE_vTaskDelete 1
#define INCLUDE_vTaskSuspend 1
#define INCLUDE_xTaskGetCurrentTaskHandle 1
#define INCLUDE_uxTaskGetStackHighWaterMark 1
#define INCLUDE_eTaskGetState 1

/* ---- Cortex-M3 interrupt priorities ---- */
#define configKERNEL_INTERRUPT_PRIORITY 255
#define configMAX_SYSCALL_INTERRUPT_PRIORITY 191

#endif /* FREERTOS_CONFIG_H */
```

FILE 3 OF 17

linker.ld

Memory map for the QEMU lm3s6965evb machine (ARM Cortex-M3). Tells the compiler where to place code, constants, and variables in the 256KB Flash / 64KB SRAM address space.

```
/* linker.ld - LM3S6965 (QEMU target: -M lm3s6965evb) */

MEMORY
{
    FLASH (rx) : ORIGIN = 0x00000000, LENGTH = 256K
    SRAM  (rwx) : ORIGIN = 0x20000000, LENGTH = 64K
}

/* Initial stack pointer = top of SRAM (stack grows downward) */
PROVIDE ( _estack = ORIGIN(SRAM) + LENGTH(SRAM) );

ENTRY(Reset_Handler)

SECTIONS
{
    .text :
    {
        KEEP(*(.isr_vector)) /* Vector table must be first in Flash */
        *(.text)
        *(.text*)
        *(.rodata)
        *(.rodata*)
        _etext = .; /* End of Flash content */
    } > FLASH

    /* .data: initialised globals - stored in Flash, copied to SRAM at startup */
    .data : AT(_etext)
    {
        _sdata = .;
        *(.data)
        *(.data*)
        _edata = .;
    } > SRAM

    /* .bss: zero-initialised globals - just zeroed in SRAM, no Flash storage */
    .bss :
    {
        _sbss = .;
        *(.bss)
        *(.bss*)
        *(COMMON)
        _ebss = .;
    } > SRAM

    ._user_heap_stack :
    {
        . = ALIGN(8);
        PROVIDE ( end = . );
        . = . + 0x400;
        . = ALIGN(8);
    } > SRAM
}
```

FILE 4 OF 17

src/startup.c

Runs before main(). Defines the interrupt vector table, copies .data from Flash to SRAM, zeros .bss, then calls main(). FreeRTOS automatically overrides SVC, PendSV, and SysTick handlers.

```
#include <stdint.h>

/* Symbols from linker.ld */
extern uint32_t _etext;
extern uint32_t _sdata, _edata;
extern uint32_t _sbss, _ebss;
extern uint32_t _estack;

void Reset_Handler(void);
void Default_Handler(void);
int main(void);

/* Weak aliases - FreeRTOS port.c overrides SVC, PendSV, SysTick */
void NMI_Handler(void)      __attribute__((weak, alias("Default_Handler")));
void HardFault_Handler(void) __attribute__((weak, alias("Default_Handler")));
void MemManage_Handler(void) __attribute__((weak, alias("Default_Handler")));
void BusFault_Handler(void) __attribute__((weak, alias("Default_Handler")));
void UsageFault_Handler(void) __attribute__((weak, alias("Default_Handler")));
void SVC_Handler(void)      __attribute__((weak, alias("Default_Handler")));
void DebugMon_Handler(void) __attribute__((weak, alias("Default_Handler")));
void PendSV_Handler(void)   __attribute__((weak, alias("Default_Handler")));
void SysTick_Handler(void)  __attribute__((weak, alias("Default_Handler")));

/* Interrupt vector table - MUST be first in Flash (see linker.ld) */
__attribute__((section(".isr_vector")))
uint32_t isr_vectors[] = {
    (uint32_t)&_estack,          /* 0: Initial stack pointer */
    (uint32_t)&Reset_Handler,     /* 1: Reset */
    (uint32_t)&NMI_Handler,       /* 2: NMI */
    (uint32_t)&HardFault_Handler, /* 3: Hard Fault */
    (uint32_t)&MemManage_Handler, /* 4: Memory Management */
    (uint32_t)&BusFault_Handler,  /* 5: Bus Fault */
    (uint32_t)&UsageFault_Handler, /* 6: Usage Fault */
    0, 0, 0, 0,                  /* 7-10: Reserved */
    (uint32_t)&SVC_Handler,        /* 11: SVC (FreeRTOS) */
    (uint32_t)&DebugMon_Handler,   /* 12: Debug Monitor */
    0,                            /* 13: Reserved */
    (uint32_t)&PendSV_Handler,     /* 14: PendSV (FreeRTOS) */
    (uint32_t)&SysTick_Handler,    /* 15: SysTick (FreeRTOS) */
};

void Reset_Handler(void) {
    uint32_t *src, *dst;

    /* Copy .data from Flash to SRAM */
    src = &_etext;
    dst = &_sdata;
    while (dst < &_edata) { *dst++ = *src++; }

    /* Zero .bss */
    dst = &_sbss;
    while (dst < &_ebss) { *dst++ = 0; }

    main();
}
```

```
    for (;;) {}  
}  
  
void Default_Handler(void) {  
    for (;;) {} /* Hang - attach GDB to see which IRQ fired */  
}
```

FILE 5 OF 17

src/drivers/uart_driver.h

Public API for the UART driver. Include this wherever you need to print debug output or send/receive data.

```
#ifndef UART_DRIVER_H
#define UART_DRIVER_H

#include <stdint.h>
#include <stddef.h>

void uart_init(uint32_t baud_rate);
void uart_putc(char c);
void uart_puts(const char *s);
void uart_printf(const char *fmt, ...); /* Thread-safe formatted print */
void uart_write_bytes(const uint8_t *data, size_t len); /* Binary TX */
int  uart_getc(uint8_t *out, uint32_t timeout_ms);      /* Returns 1 if byte received */

#endif
```


FILE 6 OF 17

src/drivers/uart_driver.c

Hardware UART driver targeting UART0 on the LM3S6965 (QEMU). Uses volatile register pointers for hardware access. Thread-safe via FreeRTOS mutex once the scheduler is running.

```
#include "uart_driver.h"
#include "FreeRTOS.h"
#include "semphr.h"
#include <stdarg.h>
#include <stdio.h>

/* LM3S6965 UART0 registers (QEMU emulates these) */
#define UART0_BASE 0x4000C000UL
#define UART0_DR    (*(volatile uint32_t *) (UART0_BASE + 0x000)) /* Data */
#define UART0_FR    (*(volatile uint32_t *) (UART0_BASE + 0x018)) /* Flags */
#define UART0_IBRD  (*(volatile uint32_t *) (UART0_BASE + 0x024)) /* Baud int */
#define UART0_FBRD  (*(volatile uint32_t *) (UART0_BASE + 0x028)) /* Baud frac */
#define UART0_LCRH  (*(volatile uint32_t *) (UART0_BASE + 0x02C)) /* Line ctrl */
#define UART0_CTL   (*(volatile uint32_t *) (UART0_BASE + 0x030)) /* Control */

#define UART_FR_TXFF (1u << 5) /* TX FIFO full */
#define UART_FR_RXFE (1u << 4) /* RX FIFO empty */
#define UART_CTL_UARTEN (1u << 0)
#define UART_CTL_TXE (1u << 8)
#define UART_CTL_RXE (1u << 9)
#define UART_LCRH_WLEN8 (0x3u << 5)
#define UART_LCRH_FEN (1u << 4)

static SemaphoreHandle_t uart_mutex = NULL;

void uart_init(uint32_t baud_rate) {
    UART0_CTL = 0;
    uint32_t clk = 16000000UL;
    UART0_IBRD = clk / (16 * baud_rate);
    UART0_FBRD = (uint32_t) (((clk % (16 * baud_rate)) * 64
        + (8 * baud_rate)) / (16 * baud_rate));
    UART0_LCRH = UART_LCRH_WLEN8 | UART_LCRH_FEN;
    UART0_CTL = UART_CTL_UARTEN | UART_CTL_TXE | UART_CTL_RXE;
    uart_mutex = xSemaphoreCreateMutex();
}

void uart_putc(char c) {
    while (UART0_FR & UART_FR_TXFF) {}
    UART0_DR = (uint32_t)c;
}

void uart_puts(const char *s) {
    while (*s) uart_putc(*s++);
}

void uart_printf(const char *fmt, ...) {
    char buf[256];
    va_list args;
    va_start(args, fmt);
    vsnprintf(buf, sizeof(buf), fmt, args);
    va_end(args);
    if (uart_mutex) xSemaphoreTake(uart_mutex, portMAX_DELAY);
    uart_puts(buf);
    if (uart_mutex) xSemaphoreGive(uart_mutex);
}
```

```
}

void uart_write_bytes(const uint8_t *data, size_t len) {
    if (uart_mutex) xSemaphoreTake(uart_mutex, portMAX_DELAY);
    for (size_t i = 0; i < len; i++) {
        while (UART0_FR & UART_FR_TXFF) {}
        UART0_DR = data[i];
    }
    if (uart_mutex) xSemaphoreGive(uart_mutex);
}

int uart_getc(uint8_t *out, uint32_t timeout_ms) {
    uint32_t deadline = xTaskGetTickCount() + pdMS_TO_TICKS(timeout_ms);
    while (xTaskGetTickCount() < deadline) {
        if (!(UART0_FR & UART_FR_RXFE)) {
            *out = (uint8_t)(UART0_DR & 0xFF);
            return 1;
        }
        vTaskDelay(pdMS_TO_TICKS(1));
    }
    return 0;
}
```

FILE 7 OF 17

src/drivers/watchdog.h

Watchdog timer API. Call watchdog_init() once at startup, then watchdog_kick() from the health monitor task every cycle to prevent system reset.

```
#ifndef WATCHDOG_H
#define WATCHDOG_H

void watchdog_init(void); /* Call once before starting scheduler */
void watchdog_kick(void); /* Call from health_monitor every 100ms */

#endif
```

FILE 8 OF 17

src/drivers/watchdog.c

Software watchdog using a FreeRTOS one-shot timer. If watchdog_kick() is not called within 2 seconds, the expire callback fires and halts the system — simulating a hardware reset.

```
#include "watchdog.h"
#include "FreeRTOS.h"
#include "timers.h"
#include "uart_driver.h"

#define WDT_TIMEOUT_MS 2000

static TimerHandle_t wdt_timer = NULL;

static void wdt_expire_cb(TimerHandle_t xTimer) {
    (void)xTimer;
    /* In production this triggers NVIC_SystemReset() */
    /* In simulation: log the fault and halt */
    uart_puts("\r\n[WDT] WATCHDOG EXPIRED — system reset!\r\n");
    taskDISABLE_INTERRUPTS();
    for (;;) {}
}

void watchdog_init(void) {
    wdt_timer = xTimerCreate(
        "WDT",
        pdMS_TO_TICKS(WDT_TIMEOUT_MS),
        pdFALSE, /* One-shot — must be reset each time */
        NULL,
        wdt_expire_cb
    );
    configASSERT(wdt_timer != NULL);
    xTimerStart(wdt_timer, 0);
}

void watchdog_kick(void) {
    xTimerReset(wdt_timer, 0); /* Restart the 2-second countdown */
}
```

FILE 9 OF 17

src/main.c

Entry point. Initialises hardware, creates queues, spawns all four tasks, then starts the FreeRTOS scheduler. Also defines the three required FreeRTOS hook functions.

```
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include "uart_driver.h"
#include "flight_types.h"
#include "watchdog.h"

/* Task priorities - higher number = higher priority */
#define PRIORITY_TELEMETRY 2
#define PRIORITY_COMMAND 3
#define PRIORITY_ADCS 4
#define PRIORITY_HEALTH 5

/* Stack sizes in WORDS (1 word = 4 bytes) */
#define STACK_ADCS 512
#define STACK_TELEMETRY 256
#define STACK_HEALTH 256
#define STACK_COMMAND 256

/* Global handles - extern'd by task files that need them */
QueueHandle_t g_telemetry_queue = NULL;
QueueHandle_t g_command_queue = NULL;
TaskHandle_t g_adcs_task_handle = NULL;

/* Forward declarations */
void adcs_task(void *p);
void telemetry_task(void *p);
void health_monitor_task(void *p);
void command_handler_task(void *p);

int main(void) {
    uart_init(115200);
    uart_puts("\r\n=== Satellite Flight Computer Booting ===\r\n");

    /* Create inter-task queues BEFORE creating tasks */
    g_telemetry_queue = xQueueCreate(16, sizeof(TelemetryPacket_t));
    g_command_queue = xQueueCreate(8, sizeof(uint8_t));
    configASSERT(g_telemetry_queue != NULL);
    configASSERT(g_command_queue != NULL);

    watchdog_init();

    /* Spawn tasks */
    configASSERT(xTaskCreate(adcs_task, "ADCS", STACK_ADCS, NULL,
        PRIORITY_ADCS, &g_adcs_task_handle) == pdPASS);
    configASSERT(xTaskCreate(telemetry_task, "TLM", STACK_TELEMETRY, NULL,
        PRIORITY_TELEMETRY, NULL) == pdPASS);
    configASSERT(xTaskCreate(health_monitor_task, "HLTH", STACK_HEALTH, NULL,
        PRIORITY_HEALTH, NULL) == pdPASS);
    configASSERT(xTaskCreate(command_handler_task, "CMD", STACK_COMMAND, NULL,
        PRIORITY_COMMAND, NULL) == pdPASS);

    uart_puts("Tasks created. Starting scheduler...\r\n");
    vTaskStartScheduler(); /* Never returns */
}
```

```
    uart_puts("FATAL: Scheduler failed!\r\n");
    for (;;) {}
}

/* ---- Required FreeRTOS hook functions ---- */

void vApplicationIdleHook(void) {
    /* Low-power sleep on real hardware: __WFI(); */
    /* Leave empty for QEMU simulation */
}

void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName) {
    (void)xTask;
    uart_printf("FATAL: Stack overflow in task [%s]\r\n", pcTaskName);
    for (;;) {}
}

void vApplicationMallocFailedHook(void) {
    uart_puts("FATAL: Heap allocation failed. Increase configTOTAL_HEAP_SIZE.\r\n");
    for (;;) {}
}
```

FILE 10 OF 17

src/tasks/adcs_task.c

Attitude Determination & Control System. Runs at exactly 50 Hz using vTaskDelayUntil. Simulates IMU sensor readings and integrates to produce roll/pitch/yaw. Publishes state via a mutex-protected global.

```
#include "FreeRTOS.h"
#include "task.h"
#include "semphr.h"
#include "flight_types.h"
#include "uart_driver.h"
#include <math.h>
#include <string.h>

/* Shared state - mutex-protected, read by telemetry and health monitor */
ADCSState_t      g_adcs_state;
SemaphoreHandle_t g_adcs_mutex = NULL;

/* Heartbeat counter - health monitor checks this is incrementing */
volatile uint32_t g_adcs_heartbeat = 0;

static void simulate_sensors(ADCSState_t *s) {
    static float t = 0.0f;
    t += 0.02f; /* 20 ms per call */

    /* Simulate a slow tumbling satellite with natural damping */
    s->omega[0] = 0.5f * sinf(t * 0.10f);
    s->omega[1] = 0.3f * cosf(t * 0.07f);
    s->omega[2] = 0.15f * sinf(t * 0.05f);

    /* Euler integration (simplified - real ADCS uses quaternions) */
    const float RAD_TO_DEG = 57.2957795f;
    s->euler[0] += s->omega[0] * 0.02f * RAD_TO_DEG;
    s->euler[1] += s->omega[1] * 0.02f * RAD_TO_DEG;
    s->euler[2] += s->omega[2] * 0.02f * RAD_TO_DEG;

    /* Wrap angles to [-180, 180] */
    for (int i = 0; i < 3; i++) {
        while (s->euler[i] > 180.0f) s->euler[i] -= 360.0f;
        while (s->euler[i] < -180.0f) s->euler[i] += 360.0f;
    }
    s->timestamp_ms = xTaskGetTickCount();
}

void adcs_task(void *p) {
    (void)p;
    g_adcs_mutex = xSemaphoreCreateMutex();
    configASSERT(g_adcs_mutex != NULL);
    memset(&g_adcs_state, 0, sizeof(g_adcs_state));
    g_adcs_state.mode = MODE_NOMINAL;

    TickType_t xLastWake = xTaskGetTickCount();
    const TickType_t xPeriod = pdMS_TO_TICKS(20); /* 50 Hz */

    for (;;) {
        /* vTaskDelayUntil = deterministic 50 Hz regardless of exec time */
        vTaskDelayUntil(&xLastWake, xPeriod);

        ADCSState_t new_state;
        simulate_sensors(&new_state);
    }
}
```

```
new_state.mode = g_adcs_state.mode; /* Preserve commanded mode */

if (xSemaphoreTake(g_adcs_mutex, pdMS_TO_TICKS(5)) == pdTRUE) {
    g_adcs_state = new_state;
    xSemaphoreGive(g_adcs_mutex);
}

g_adcs_heartbeat++; /* Health monitor watches this */
}
```


FILE 11 OF 17

src/tasks/telemetry_task.c

Downlink task running at 1 Hz. Reads the ADCS state, packs it into a binary TelemetryPacket_t with magic number and XOR checksum, and transmits it over UART to the ground station.

```
#include "FreeRTOS.h"
#include "task.h"
#include "flight_types.h"
#include "uart_driver.h"

extern ADCSState_t      g_adcs_state;
extern SemaphoreHandle_t g_adcs_mutex;

static uint16_t xor_checksum(const uint8_t *data, size_t len) {
    uint16_t c = 0;
    for (size_t i = 0; i < len; i++) c ^= data[i];
    return c;
}

void telemetry_task(void *p) {
    (void)p;
    static uint16_t seq = 0;
    TelemetryPacket_t pkt;

    for (;;) {
        vTaskDelay(pdMS_TO_TICKS(1000)); /* 1 Hz downlink */

        if (xSemaphoreTake(g_adcs_mutex, pdMS_TO_TICKS(10)) == pdTRUE) {
            pkt.magic      = TLM_MAGIC;
            pkt.sequence    = seq++;
            pkt.timestamp_ms = g_adcs_state.timestamp_ms;
            pkt.roll_deg    = g_adcs_state.euler[0];
            pkt.pitch_deg   = g_adcs_state.euler[1];
            pkt.yaw_deg     = g_adcs_state.euler[2];
            pkt.omega_x     = g_adcs_state.omega[0];
            pkt.omega_y     = g_adcs_state.omega[1];
            pkt.omega_z     = g_adcs_state.omega[2];
            pkt.mode        = g_adcs_state.mode;
            pkt.fault_flags = 0;
            pkt.temperature = 2350; /* 23.5 C */
            pkt.checksum    = 0;
            pkt.checksum    = xor_checksum((uint8_t *)&pkt, sizeof(pkt) - 2);
            xSemaphoreGive(g_adcs_mutex);
        }

        uart_write_bytes((uint8_t *)&pkt, sizeof(pkt));
    }
}
```

FILE 12 OF 17

src/tasks/health_monitor.c

Runs at 10 Hz with the highest priority. Checks the ADCS heartbeat, monitors stack usage, updates the fault flag bitmask, and kicks the watchdog. If ADCS stops responding it sets FAULT_ADCS_TIMEOUT.

```
#include "FreeRTOS.h"
#include "task.h"
#include "flight_types.h"
#include "watchdog.h"
#include "uart_driver.h"

extern volatile uint32_t g_adcs_heartbeat;
extern TaskHandle_t      g_adcs_task_handle;

volatile uint8_t g_fault_flags = FAULT_NONE;

void health_monitor_task(void *p) {
    (void)p;
    TickType_t xLastWake = xTaskGetTickCount();
    const TickType_t xPeriod = pdMS_TO_TICKS(100); /* 10 Hz */
    uint32_t last_hb = 0;

    for (;;) {
        vTaskDelayUntil(&xLastWake, xPeriod);

        /* --- Check ADCS heartbeat --- */
        uint32_t current_hb = g_adcs_heartbeat;
        if (current_hb == last_hb) {
            g_fault_flags |= FAULT_ADCS_TIMEOUT;
            uart_puts("[HM] FAULT: ADCS heartbeat missed!\r\n");
        } else {
            g_fault_flags &= ~FAULT_ADCS_TIMEOUT;
        }
        last_hb = current_hb;

        /* --- Check ADCS stack high-water mark --- */
        if (g_adcs_task_handle != NULL) {
            UBaseType_t watermark = uxTaskGetStackHighWaterMark(g_adcs_task_handle);
            if (watermark < 32) { /* < 128 bytes remaining */
                g_fault_flags |= FAULT_STACK_LOW;
                uart_printf("[HM] WARNING: ADCS stack low: %u words left\r\n",
                           (unsigned)watermark);
            } else {
                g_fault_flags &= ~FAULT_STACK_LOW;
            }
        }

        /* --- Kick watchdog (confirms health monitor is alive) --- */
        watchdog_kick();

        /* --- Periodic status log (every 10 cycles = 1 second) --- */
        static uint8_t log_divider = 0;
        if (++log_divider >= 10) {
            log_divider = 0;
            uart_printf("[HM] tick=%u faults=0x%02X adcs_hb=%u\r\n",
                       (unsigned)xTaskGetTickCount(),
                       g_fault_flags,
                       (unsigned)current_hb);
        }
    }
}
```

```
}  
}
```

FILE 13 OF 17

src/tasks/command_handler.c

Receives single-byte commands from the ground station GUI via UART. Dispatches mode changes by writing to the shared ADCS state under mutex protection. Command bytes must match gui.py.

```
#include "FreeRTOS.h"
#include "task.h"
#include "flight_types.h"
#include "uart_driver.h"

extern ADCSState_t      g_adcs_state;
extern SemaphoreHandle_t g_adcs_mutex;

#define CMD_NOMINAL      'N'
#define CMD_SAFE         'S'
#define CMD_DETUMBLE     'D'
#define CMD_REBOOT       'R'
#define CMD_STATUS       '?'

void command_handler_task(void *p) {
    (void)p;
    uint8_t byte;
    uart_puts("[CMD] Ready. Listening for uplink commands...\r\n");

    for (;;) {
        if (uart_getc(&byte, 1000) == 0) continue; /* 1s timeout, retry */

        uart_printf("[CMD] Rx: 0x%02X ('%c')\r\n", byte, byte);

        switch ((char)byte) {
            case CMD_NOMINAL:
                if (xSemaphoreTake(g_adcs_mutex, pdMS_TO_TICKS(10)) == pdTRUE) {
                    g_adcs_state.mode = MODE_NOMINAL;
                    xSemaphoreGive(g_adcs_mutex);
                    uart_puts("[CMD] Mode -> NOMINAL\r\n");
                }
                break;
            case CMD_SAFE:
                if (xSemaphoreTake(g_adcs_mutex, pdMS_TO_TICKS(10)) == pdTRUE) {
                    g_adcs_state.mode = MODE_SAFE;
                    xSemaphoreGive(g_adcs_mutex);
                    uart_puts("[CMD] Mode -> SAFE\r\n");
                }
                break;
            case CMD_DETUMBLE:
                if (xSemaphoreTake(g_adcs_mutex, pdMS_TO_TICKS(10)) == pdTRUE) {
                    g_adcs_state.mode = MODE_DETUMBLE;
                    xSemaphoreGive(g_adcs_mutex);
                    uart_puts("[CMD] Mode -> DETUMBLE\r\n");
                }
                break;
            case CMD_STATUS:
                uart_printf("[CMD] mode=%u tick=%u faults=0x%02X\r\n",
                           g_adcs_state.mode,
                           (unsigned)xTaskGetTickCount(),
                           0);
                break;
            case CMD_REBOOT:
                uart_puts("[CMD] Reboot commanded.\r\n");
                vTaskSuspendAll();
        }
    }
}
```

```
        for (;;) {}
    default:
        uart_printf("[CMD] Unknown: 0x%02X\r\n", byte);
    }
}
```

FILE 14 OF 17

CMakeLists.txt

Root build file. Sets up the ARM cross-compiler, lists every source file, configures include paths, and generates .elf / .bin / .hex output files.

```
cmake_minimum_required(VERSION 3.18)

# Cross-compiler setup - must come BEFORE project()
set(CMAKE_SYSTEM_NAME      Generic)
set(CMAKE_SYSTEM_PROCESSOR arm)
set(CMAKE_TRY_COMPILE_TARGET_TYPE STATIC_LIBRARY)
set(CMAKE_C_COMPILER        arm-none-eabi-gcc)
set(CMAKE_OBJCOPY           arm-none-eabi-objcopy)

project(SatelliteFC C)

set(CPU_FLAGS "-mcpu=cortex-m3 -mthumb -mfloat-abi=soft")

set(CMAKE_C_FLAGS
    "${CPU_FLAGS} -O2 -Wall -Wextra -Wno-unused-parameter"
    " -ffunction-sections -fdata-sections -fno-common")

set(CMAKE_EXE_LINKER_FLAGS
    "${CPU_FLAGS}"
    " -T ${CMAKE_SOURCE_DIR}/linker.ld"
    " -nostdlib --specs=nosys.specs"
    " -Wl,--gc-sections"
    " -Wl,-Map=satellite_fc.map"
    " -lm")

# FreeRTOS kernel
set(FREERTOS_SRC
    FreeRTOS/Source/tasks.c
    FreeRTOS/Source/queue.c
    FreeRTOS/Source/list.c
    FreeRTOS/Source/timers.c
    FreeRTOS/Source/event_groups.c
    FreeRTOS/Source/portable/GCC/ARM_CM3/port.c
    FreeRTOS/Source/portable/MemMang/heap_4.c)

# Application
set(APP_SRC
    src/startup.c
    src/main.c
    src/tasks/adcs_task.c
    src/tasks/telemetry_task.c
    src/tasks/health_monitor.c
    src/tasks/command_handler.c
    src/drivers/uart_driver.c
    src/drivers/watchdog.c)

add_executable(satellite_fc.elf ${APP_SRC} ${FREERTOS_SRC})

target_include_directories(satellite_fc.elf PRIVATE
    src/common
    src
    src/drivers
    src/tasks
    FreeRTOS/Source/include
```

FreeRTOS/Source/portable/GCC/ARM_CM3)

```
add_custom_command(TARGET satellite_fc.elf POST_BUILD
  COMMAND ${CMAKE_OBJCOPY} -O binary satellite_fc.elf satellite_fc.bin
  COMMAND ${CMAKE_OBJCOPY} -O ihex    satellite_fc.elf satellite_fc.hex
  COMMAND arm-none-eabi-size satellite_fc.elf
  COMMENT "Post-build: binary, hex, and size")
```

FILE 15 OF 17

ground_station/telemetry_parser.py

Python module that parses raw binary UART bytes into telemetry packet dicts. The struct.unpack format string MUST match TelemetryPacket_t in flight_types.h byte-for-byte.

```
import struct

TLM_MAGIC = 0xA5C3
TLM_FORMAT = "<HHIffffffBBhH" # little-endian packed - matches TelemetryPacket_t
TLM_SIZE = struct.calcsize(TLM_FORMAT) # 46 bytes

MODES = {0: "SAFE", 1: "DETUMBLE", 2: "NADIR", 3: "NOMINAL"}
FIELDS = ["magic", "sequence", "timestamp_ms",
          "roll", "pitch", "yaw",
          "omega_x", "omega_y", "omega_z",
          "mode", "faults", "temperature", "checksum"]

def parse_packet(data: bytes) -> dict | None:
    """Parse bytes -> dict. Returns None if magic or checksum is invalid."""
    if len(data) < TLM_SIZE:
        return None
    try:
        values = struct.unpack(TLM_FORMAT, data[:TLM_SIZE])
        pkt = dict(zip(FIELDS, values))
        if pkt["magic"] != TLM_MAGIC:
            return None
        # Verify XOR checksum
        csum = 0
        for byte in data[:TLM_SIZE - 2]:
            csum ^= byte
        if csum != pkt["checksum"]:
            return None
        pkt["temperature_c"] = pkt["temperature"] / 10.0
        pkt["mode_str"] = MODES.get(pkt["mode"], "UNKNOWN")
        return pkt
    except struct.error:
        return None

def find_packet(buf: bytearray) -> tuple:
    """Scan a bytearray for a valid packet.
    Returns (packet_dict_or_None, bytes_consumed).
    Caller should remove the consumed bytes from buf."""
    magic_bytes = struct.pack("<H", TLM_MAGIC)
    idx = buf.find(magic_bytes)
    if idx < 0:
        return None, len(buf) # No magic found - discard everything
    pkt = parse_packet(bytes(buf[idx:]))
    if pkt:
        return pkt, idx + TLM_SIZE # Good packet
    return None, idx + 1 # Bad packet - skip one byte and retry
```


FILE 16 OF 17

ground_station/gui.py

Tkinter + matplotlib ground station GUI. Dark-themed dashboard shows live roll/pitch/yaw plots, all telemetry fields, active fault flags, and command uplink buttons. Connects to flight computer via serial port.

```
import tkinter as tk
import serial, threading, queue
import matplotlib.pyplot as plt
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
from collections import deque
from telemetry_parser import find_packet

MAX_HISTORY = 120 # 2 minutes at 1 Hz

class GroundStation:
    def __init__(self, port="/tmp/gs_port", baud=115200):
        self.port = port
        self.baud = baud
        self.running = False
        self.q = queue.Queue(maxsize=100)
        self.times = deque(maxlen=MAX_HISTORY)
        self.rolls = deque(maxlen=MAX_HISTORY)
        self.pitches = deque(maxlen=MAX_HISTORY)
        self.yaws = deque(maxlen=MAX_HISTORY)
        self.pkt_count = 0
        self._build_ui()

    def _build_ui(self):
        self.root = tk.Tk()
        self.root.title("Satellite Ground Station")
        self.root.configure(bg="#0D1B2A")
        self.root.geometry("1200x760")

        # Header bar
        hdr = tk.Frame(self.root, bg="#0078D4", height=48)
        hdr.pack(fill=tk.X)
        tk.Label(hdr, text="SATELLITE FLIGHT COMPUTER | GROUND STATION",
                  font=("Arial",13,"bold"), bg="#0078D4", fg="white").pack(side=tk.LEFT,
padx=16, pady=10)
        self.status_lbl = tk.Label(hdr, text="OFFLINE",
                                   font=("Arial",12,"bold"), bg="#0078D4", fg="#FF6B6B")
        self.status_lbl.pack(side=tk.RIGHT, padx=16)

        # Layout
        main = tk.Frame(self.root, bg="#0D1B2A")
        main.pack(fill=tk.BOTH, expand=True, padx=8, pady=8)

        left = tk.Frame(main, bg="#0D1B2A", width=300)
        left.pack(side=tk.LEFT, fill=tk.Y, padx=(0, 8))

        self._card(left, "ATTITUDE (deg)",
                  ["Roll","Pitch","Yaw"],
                  ["roll_lbl","pitch_lbl","yaw_lbl"])
        self._card(left, "ANGULAR RATE (rad/s)",
                  ["Omega X","Omega Y","Omega Z"],
                  ["ox_lbl","oy_lbl","oz_lbl"])
        self._card(left, "SYSTEM",
                  ["Mode","Faults","Temp (C)","Packets"],
                  ["mode_lbl","fault_lbl","temp_lbl","pkt_lbl"])
```

```

# Command panel
cmd_frame = tk.LabelFrame(left, text="COMMAND UPLINK",
                           fg="#0078D4", bg="#132436",
                           font=("Arial",10,"bold"))
cmd_frame.pack(fill=tk.X, pady=4)
commands = [("NOMINAL MODE", "N"), ("SAFE MODE", "S"),
            ("DETUMBLE MODE", "D"), ("REBOOT", "R")]
for label, byte in commands:
    tk.Button(cmd_frame, text=label,
              command=lambda b=byte: self._send(b),
              bg="#1B3A5C", fg="white",
              font=("Courier",9,"bold"),
              relief=tk.FLAT, padx=8, pady=4
              ).pack(fill=tk.X, padx=4, pady=2)

# Plot panel
right = tk.Frame(main, bg="#0D1B2A")
right.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

plt.style.use("dark_background")
self.fig, (self.ax1, self.ax2) = plt.subplots(2, 1, figsize=(7, 5))
self.fig.patch.set_facecolor("#0D1B2A")
for ax in (self.ax1, self.ax2):
    ax.set_facecolor("#132436")

self.canvas = FigureCanvasTkAgg(self.fig, master=right)
self.canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)

self.conn_btn = tk.Button(
    self.root, text="CONNECT",
    command=self._toggle,
    bg="#107C10", fg="white",
    font=("Arial",11,"bold"), pady=6)
self.conn_btn.pack(fill=tk.X, padx=8, pady=(0, 8))

def _card(self, parent, title, labels, attr_names):
    frame = tk.LabelFrame(parent, text=title,
                           fg="#0078D4", bg="#132436",
                           font=("Arial",10,"bold"))
    frame.pack(fill=tk.X, pady=3)
    for lbl, attr in zip(labels, attr_names):
        row = tk.Frame(frame, bg="#132436")
        row.pack(fill=tk.X, padx=6)
        tk.Label(row, text=lbl+":", width=12, anchor="w",
                  bg="#132436", fg="#8899AA",
                  font=("Courier",10)).pack(side=tk.LEFT)
        val = tk.Label(row, text="--", anchor="w",
                        bg="#132436", fg="#00FF88",
                        font=("Courier",10,"bold"))
        val.pack(side=tk.LEFT)
        setattr(self, attr, val)

def _toggle(self):
    if self.running:
        self.running = False
        self.conn_btn.config(text="CONNECT", bg="#107C10")
        self.status_lbl.config(text="OFFLINE", fg="#FF6B6B")
    else:
        self.running = True
        self.conn_btn.config(text="DISCONNECT", bg="#D83B01")
        self.status_lbl.config(text="ONLINE", fg="#00FF88")
        threading.Thread(target=self._rx, daemon=True).start()
        self._tick()

```

```

def _rx(self):
    """Background thread: read serial bytes, find packets, push to queue."""
    try:
        ser = serial.Serial(self.port, self.baud, timeout=1)
        buf = bytearray()
        while self.running:
            data = ser.read(256)
            if data:
                buf.extend(data)
                pkt, consumed = find_packet(buf)
                buf = buf[consumed:]
                if pkt and not self.q.full():
                    self.q.put(pkt)
    except serial.SerialException as e:
        print(f"Serial error: {e}")
        self.running = False

def _tick(self):
    """Main thread: drain queue, update labels and plot every 500 ms."""
    while not self.q.empty():
        self._update(self.q.get_nowait())
    if self.running:
        self.root.after(500, self._tick)

def _update(self, pkt):
    self.pkt_count += 1
    self.times.append(pkt["timestamp_ms"] / 1000.0)
    self.rolls.append(pkt["roll"])
    self.pitches.append(pkt["pitch"])
    self.yaws.append(pkt["yaw"])

    self.roll_lbl.config(text=f'{ pkt["roll"]:+8.2f}')
    self.pitch_lbl.config(text=f'{ pkt["pitch"]:+8.2f}')
    self.yaw_lbl.config(text=f'{ pkt["yaw"]:+8.2f}')
    self.ox_lbl.config(text=f'{ pkt["omega_x"]:+7.4f}')
    self.oy_lbl.config(text=f'{ pkt["omega_y"]:+7.4f}')
    self.oz_lbl.config(text=f'{ pkt["omega_z"]:+7.4f}')
    self.mode_lbl.config(text=pkt["mode_str"])
    self.temp_lbl.config(text=f'{pkt["temperature_c"]:.1f}')
    self.pkt_lbl.config(text=str(self.pkt_count))

    fault_color = "#00FF88" if pkt["faults"] == 0 else "#FF4444"
    fault_text = "NOMINAL" if pkt["faults"] == 0 else f'0x{pkt["faults"]:02X}'
    self.fault_lbl.config(text=fault_text, fg=fault_color)

    xs = list(self.times)
    self.ax1.cla()
    self.ax2.cla()
    self.ax1.plot(xs, list(self.rolls), color="#FF6B6B", label="Roll")
    self.ax1.plot(xs, list(self.pitches), color="#4ECDC4", label="Pitch")
    self.ax1.plot(xs, list(self.yaws), color="#FFE66D", label="Yaw")
    self.ax1.set_title("Attitude (deg)", color="white")
    self.ax1.legend(loc="upper right", fontsize=8)
    self.ax1.set_facecolor("#132436")
    self.ax2.plot(xs, list(deque([p["omega_x"] for p in []], maxlen=MAX_HISTORY)),
                  color="#A8D8EA", label="Omega X")
    self.ax2.set_title("Angular Rate (rad/s)", color="white")
    self.ax2.set_facecolor("#132436")
    self.canvas.draw()

def _send(self, cmd: str):
    """Send a command byte to the flight computer."""
    print(f"CMD -> {cmd}") # Extend: write cmd.encode() to serial

def run(self):

```

```
        self.root.mainloop()

if __name__ == "__main__":
    import argparse
    ap = argparse.ArgumentParser()
    ap.add_argument("--port", default="/tmp/gs_port")
    ap.add_argument("--baud", type=int, default=115200)
    args = ap.parse_args()
    GroundStation(args.port, args.baud).run()
```

FILE 17 OF 17

tests/test_parser.py

pytest unit tests for the telemetry parser. Tests valid packets, bad magic numbers, corrupted checksums, and short buffers. Run with: pytest tests/ -v

```
import struct, pytest, sys, os
sys.path.insert(0, os.path.join(os.path.dirname(__file__), "..", "ground_station"))
from telemetry_parser import parse_packet, find_packet, TLM_MAGIC, TLM_FORMAT, TLM_SIZE

def _make(magic=TLM_MAGIC, roll=10.5, pitch=-5.2, yaw=90.0,
          mode=3, faults=0, seq=1, ts=1000,
          ox=0.1, oy=0.0, oz=-0.05, temp=2350) -> bytes:
    """Build a valid binary packet with correct checksum."""
    raw = struct.pack(TLM_FORMAT, magic, seq, ts,
                     roll, pitch, yaw, ox, oy, oz,
                     mode, faults, temp, 0)

    csum = 0
    for b in raw[:-2]: csum ^= b
    return raw[:-2] + struct.pack("<H", csum)

def test_valid_packet():
    pkt = parse_packet(_make())
    assert pkt is not None
    assert pkt["roll"] == pytest.approx(10.5, abs=1e-4)
    assert pkt["pitch"] == pytest.approx(-5.2, abs=1e-4)
    assert pkt["yaw"] == pytest.approx(90.0, abs=1e-4)
    assert pkt["mode_str"] == "NOMINAL"
    assert pkt["temperature_c"] == pytest.approx(235.0, abs=0.01)

def test_bad_magic_rejected():
    assert parse_packet(_make(magic=0xDEAD)) is None

def test_bad_checksum_rejected():
    raw = bytearray(_make())
    raw[-1] ^= 0xFF # Flip bits in checksum
    assert parse_packet(bytes(raw)) is None

def test_too_short_returns_none():
    assert parse_packet(b"\xC3\xA5\x00") is None

def test_find_packet_with_garbage_prefix():
    garbage = b"\xDE\xAD\xBE\xEF" * 3
    buf = bytearray(garbage + _make())
    pkt, consumed = find_packet(buf)
    assert pkt is not None
    assert pkt["roll"] == pytest.approx(10.5, abs=1e-4)

def test_find_packet_incomplete_returns_none():
    half = bytearray(_make()[:10]) # Only half a packet
    pkt, _ = find_packet(half)
    assert pkt is None
```

All 17 files shown. Nothing omitted.

Use Document 4 (Step-by-Step Process Guide) to know exactly when and in what order to create each file.